

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ
по лабораторной работе №9
по дисциплине «Вычислительная математика»
Тема: Составные формулы прямоугольников, трапеций, Симпсона.

Студент гр. 4342

Тиссен М.Ю.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Цель лабораторной работы - изучить численное вычисление определенного интеграла с помощью составных квадратурных формул прямоугольников, трапеций и Симпсона. В ходе работы требуется реализовать подынтегральную функцию варианта, вычислить значение интеграла каждым из указанных методов, а также применить правило Рунге для оценки погрешности результата. Дополнительно необходимо организовать ввод требуемой точности через запуск программы, подобрать для каждого метода такое количество разбиений n , при котором оценка погрешности не превышает заданное значение ε , и сравнить полученные результаты между собой по точности и количеству необходимых разбиений.

Теоретический минимум.

Численное интегрирование применяется для приближенного вычисления определенных интегралов. Такой подход используется тогда, когда интеграл трудно найти аналитически или когда нужно получить численное значение с заданной точностью. Основная идея численного интегрирования заключается в том, что площадь под графиком функции на отрезке $[a, b]$ заменяется суммой площадей более простых геометрических фигур.

Для применения составных квадратурных формул отрезок интегрирования $[a, b]$ разбивается на n равных частей. Длина одного маленького отрезка называется шагом сетки и вычисляется по формуле:

$$h = (b - a) / n$$

Точки разбиения имеют вид:

$$x_i = a + i * h, \text{ где } i = 0, 1, 2, \dots, n$$

Чем больше количество разбиений n , тем меньше шаг h и тем точнее обычно получается приближенное значение интеграла. В лабораторной работе рассматриваются три численных метода: метод прямоугольников, метод трапеций и метод Симпсона.

В методе прямоугольников площадь под графиком функции заменяется суммой площадей прямоугольников. В данной работе используется формула средних прямоугольников. Это означает, что на каждом маленьком отрезке значение функции берется в середине этого отрезка. Составная формула средних прямоугольников имеет вид:

$$I \approx h * \sum f(a + (i + 0,5) * h), \text{ где } i = 0, 1, \dots, n - 1$$

Иначе эту формулу можно записать в развернутом виде:

$$I \approx h * (f(a + 0,5h) + f(a + 1,5h) + \dots + f(a + (n - 0,5)h))$$

Данный метод прост в реализации. Его смысл заключается в том, что на каждом участке график функции приближенно заменяется горизонтальным отрезком, а площадь под графиком считается как площадь прямоугольника.

В методе трапеций график функции на каждом маленьком отрезке приближенно заменяется прямой линией. Поэтому площадь под графиком на каждом участке считается как площадь трапеции. Составная формула трапеций имеет вид:

$$I \approx h * ((f(a) + f(b)) / 2 + f(a + h) + f(a + 2h) + \dots + f(a + (n - 1)h))$$

В этой формуле значения функции на концах всего отрезка учитываются с половинным весом, а значения функции во внутренних точках сетки учитываются с полным весом. Метод трапеций обычно дает более точное приближение, чем простые формулы левых или правых прямоугольников, так как учитывает изменение функции между соседними точками.

Метод Симпсона является более точным методом численного интегрирования. В нем график функции на парах соседних отрезков приближенно заменяется параболой. За счет этого метод Симпсона часто требует меньшее количество разбиений для достижения заданной точности. Составная формула Симпсона имеет вид:

$$I \approx h / 3 * (f(x_0) + 4f(x_1) + 2f(x_2) + 4f(x_3) + \dots + 2f(x_{(n-2)}) + 4f(x_{(n-1)}) + f(x_n))$$

Для применения формулы Симпсона количество отрезков n должно быть четным, так как метод работает на парах соседних промежутков.

Для оценки погрешности в лабораторной работе используется правило Рунге. Его смысл заключается в том, что интеграл вычисляется два раза: сначала на более грубой сетке, затем на более мелкой сетке. В работе значение интеграла на грубой сетке обозначается как I_{2h} , а значение интеграла на более мелкой сетке — как I_h .

Оценка погрешности по правилу Рунге вычисляется по формуле:

$$R = |I_h - I_{2h}| / (2^p - 1)$$

Здесь p — порядок точности используемого метода. Для формулы прямоугольников и формулы трапеций порядок точности равен 2, поэтому оценка погрешности имеет вид:

$$R = |I_h - I_{2h}| / 3$$

Для формулы Симпсона порядок точности равен 4, поэтому используется формула:

$$R = |I_h - I_{2h}| / 15$$

Вычисления продолжаются до тех пор, пока оценка погрешности не станет меньше или равна заданной точности ε :

$$R \leq \varepsilon$$

Таким образом, правило Рунге позволяет автоматически подобрать такое количество разбиений n , при котором приближенное значение интеграла будет найдено с требуемой точностью. Чем меньше заданное значение ε , тем больше разбиений обычно требуется для выполнения условия точности.

Выполнение работы.

1. Реализация функций для вычисления интеграла методами прямоугольников, трапеций и Симпсона.

В программе были реализованы три отдельных способа приближенного вычисления определенного интеграла. Для каждого метода создана своя функция, так как формулы прямоугольников, трапеций и Симпсона используют разные правила выбора точек и разные коэффициенты при значениях функции.

Общим для всех методов является то, что отрезок интегрирования делится на n равных частей. После этого вычисляется шаг сетки:

$$h = (b - a) / n$$

Далее каждый метод по-своему использует полученную сетку.

В методе прямоугольников используется вариант средних прямоугольников. Это означает, что на каждом промежутке берется не левая и не правая граница, а середина маленького отрезка. В этой точке вычисляется значение функции, после чего все значения складываются и умножаются на шаг h . Такой подход позволяет приближенно заменить площадь под графиком суммой площадей прямоугольников.

В методе трапеций значения функции берутся в узлах сетки. Значения на концах всего отрезка учитываются с коэффициентом $1/2$, а значения во внутренних точках — полностью. После суммирования результат умножается на шаг h . Геометрически это соответствует тому, что график функции на каждом маленьком промежутке заменяется отрезком прямой, а площадь находится как площадь трапеции.

В методе Симпсона используются значения функции с чередующимися коэффициентами 4 и 2. Значения на концах отрезка входят в сумму с коэффициентом 1. Перед вычислением проверяется, что число разбиений n является четным, так как формула Симпсона применяется к

парам соседних промежутков. Если n нечетное, программа выдает ошибку. Такой метод обычно дает более точный результат, потому что график функции приближается не прямыми линиями, а параболами.

Таким образом, каждая квадратурная формула была вынесена в отдельную функцию. Это делает программу более понятной и позволяет независимо проверять работу каждого метода.

2. Реализация функции для вычисления подынтегрального выражения.

Для вычисления значений подынтегральной функции в программе создана отдельная функция $f(x)$. Она получает на вход значение аргумента x и возвращает значение выражения, заданного вариантом.

Подынтегральная функция имеет вид:

$$f(x) = x^2 \cdot \exp(-x^2)$$

Для вычисления экспоненты используется функция $\exp$ из стандартного модуля `math`. Так как все методы интегрирования многократно обращаются к одному и тому же выражению, вынесение его в отдельную функцию упрощает программу. Если потребуется изменить подынтегральную функцию, достаточно будет изменить только функцию $f(x)$, не переписывая реализации методов прямоугольников, трапеций и Симпсона.

На рассматриваемом отрезке функция определена и непрерывна, поэтому ее можно использовать для численного интегрирования указанными методами.

3. Организация главной программы и оценка погрешности по правилу Рунге.

Главная часть программы отвечает за ввод точности, запуск всех методов и вывод итоговых результатов. Точность ε передается через командную строку при запуске программы. Пользователь может указать

одну точность или несколько значений сразу. Если значение точности не передано, программа сообщает об ошибке и выводит примеры правильного запуска.

Также выполняется проверка введенных данных. Если пользователь ввел не число или указал неположительную точность, программа завершает работу с сообщением об ошибке. Это сделано для того, чтобы избежать некорректных вычислений.

После чтения точности задаются границы интегрирования:

$$a = 0$$

$$b = \pi$$

Затем формируется список методов. Для каждого метода указывается его название, функция вычисления интеграла, порядок точности и знак уточнения по правилу Рунге. Для прямоугольников и трапеций используется порядок $p = 2$, а для формулы Симпсона — $p = 4$.

Подбор количества разбиений выполняется отдельно для каждого метода. Сначала берется начальное значение $n = 4$. Затем интеграл вычисляется два раза: на грубой сетке и на более мелкой сетке. Значение на грубой сетке обозначается как I_{2h} , а значение на мелкой сетке — как I_h .

После этого программа оценивает погрешность по правилу Рунге:

$$R = |I_h - I_{2h}| / (2^p - 1)$$

Если полученная оценка погрешности не превышает заданную точность ε , найденное значение n считается подходящим. Если же погрешность больше ε , количество разбиений увеличивается, и вычисления повторяются.

В программе рассматриваются значения n , кратные четырем. Это удобно для формулы Симпсона, потому что при таком подходе и мелкая, и грубая сетка имеют четное число отрезков. Значит, формула Симпсона может применяться корректно на каждом шаге проверки.

После завершения вычислений программа выводит таблицу результатов. Для каждого метода показывается количество отрезков на грубой сетке, количество отрезков на мелкой сетке, шаг h , значения I_{2h} и I_h , оценка погрешности и итоговое уточненное значение интеграла.

Дополнительно программа определяет, какому методу потребовалось наименьшее количество разбиений для достижения заданной точности. Это позволяет сравнить методы не только по полученному значению интеграла, но и по эффективности.

4. Обработка входных данных и проверка корректности точности.

В программе предусмотрен ввод требуемой точности через аргументы командной строки. Это позволяет запускать программу как для одного значения ϵ , так и сразу для нескольких значений точности. Например, пользователь может передать при запуске программы значения 0.01, 0.001 и 0.0001, после чего программа последовательно выполнит вычисления для каждого из них. Для обработки входных данных реализована отдельная функция, которая получает список аргументов запуска и преобразует их в числовые значения. Если пользователь не указал точность, программа выводит сообщение об ошибке и показывает примеры правильного запуска. Также выполняется проверка, что введенное значение действительно является числом и что оно больше нуля. Если точность задана некорректно, программа завершает работу, так как дальнейшие вычисления с таким значением не имеют смысла. Дополнительно в программе допускается ввод дробного числа как через точку, так и через запятую. Для этого перед преобразованием строки в число запятая заменяется на точку. Это делает запуск программы более удобным для пользователя. Таким образом, программа не просто

выполняет вычисления, но и контролирует корректность исходных данных, что снижает вероятность ошибок при запуске.

5. Хранение информации о методах и результатах вычислений.

Для удобной организации программы используются специальные структуры данных. Информация о каждом методе интегрирования хранится отдельно: название метода, функция, которая выполняет вычисление интеграла, порядок точности метода и знак уточнения по правилу Рунге. Такой подход позволяет не писать отдельный одинаковый код для каждого метода. Вместо этого программа проходит по списку методов и для каждого из них выполняет один и тот же общий алгоритм: вычисляет интеграл на двух сетках, оценивает погрешность, подбирает подходящее значение n и сохраняет результат. Результаты вычислений также сохраняются в отдельной структуре. В нее записываются название метода, заданная точность, количество отрезков на грубой и мелкой сетке, шаг h , значения I_{2h} и I_h , оценка погрешности по правилу Рунге и итоговое уточненное значение интеграла. Такое хранение данных делает программу более аккуратной и понятной. Все значения, относящиеся к одному методу, находятся вместе, поэтому их удобно передавать в функцию вывода и использовать для дальнейшего сравнения.

6. Вывод таблицы и сравнение методов.

После завершения вычислений программа выводит результаты в виде таблицы. Для каждой заданной точности формируется отдельная таблица, в которой представлены результаты всех трех методов: прямоугольников, трапеций и Симпсона. В таблице отображаются следующие данные: количество отрезков на грубой сетке $n/2$, количество отрезков на мелкой сетке n , шаг h , значение интеграла на грубой сетке I_{2h} , значение интеграла на мелкой сетке I_h , оценка погрешности и итоговое уточненное значение интеграла. Такой формат вывода удобен для

анализа, потому что позволяет сразу сравнить методы между собой. По таблице можно увидеть, какое значение интеграла получилось каждым методом, какая была оценка погрешности и сколько разбиений потребовалось для достижения заданной точности. Дополнительно программа определяет метод, которому потребовалось наименьшее количество отрезков n . Это позволяет сделать сравнительную оценку эффективности методов. Если метод достигает требуемой точности при меньшем числе разбиений, значит, в данной задаче он оказался более эффективным.

Выводы.

В ходе выполнения лабораторной работы были изучены и реализованы составные квадратурные формулы для численного вычисления определенного интеграла: метод прямоугольников, метод трапеций и метод Симпсона. Для каждого метода была написана отдельная функция, что позволило сравнить результаты вычислений и количество разбиений, необходимое для достижения заданной точности. Также была реализована функция для вычисления подынтегрального выражения по варианту.

В программе был выполнен автоматический подбор количества разбиений n с использованием правила Рунге. Интеграл вычислялся на грубой и мелкой сетках, после чего оценивалась погрешность. Если погрешность не превышала заданное значение ϵ_{rs} , результат считался найденным с требуемой точностью. Такой подход позволил не задавать число разбиений вручную, а подбирать его автоматически для каждого метода.

Также в программе была реализована обработка входных данных через аргументы командной строки. Это позволило задавать одну или несколько точностей при запуске программы. Дополнительно выполнялась проверка корректности введенных значений, что делает программу более удобной и устойчивой к ошибкам пользователя.

По результатам работы можно сделать вывод, что все три метода позволяют получить приближенное значение интеграла с заданной точностью, однако отличаются скоростью сходимости. Метод Симпсона обычно требует меньшее количество разбиений, так как имеет более высокий порядок точности. Методы прямоугольников и трапеций также дают корректный результат, но для достижения той же точности им может потребоваться большее значение n . Таким образом, правило Рунге позволяет удобно контролировать погрешность вычислений, а сравнение

методов показывает, какой из них является более эффективным для рассматриваемой задачи.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл main.py

```
import math
import sys
from dataclasses import dataclass
from typing import Callable, List

"""

Тема:
Численное интегрирование.
Составные квадратурные формулы прямоугольников, трапеций и Симпсона.
Оценка погрешности по правилу Рунге.

Интеграл от 0 до pi:

$$x^2 * \exp(-x^2) dx$$


Точность задается через запуск программы.

Примеры запуска:
python main.py 0.01
python main.py 0.01 0.001 0.0001

"""

# Константа pi.
PI = math.pi

def f(x: float) -> float:
    """
    Подынтегральная функция.

    По условию:

$$f(x) = x^2 * \exp(-x^2)$$


    На отрезке [0; pi] функция определена и непрерывна,
    поэтому для нее можно применять формулы прямоугольников,
    трапеций и Симпсона.
    """
```

```

"""

return x ** 2 * math.exp(-x ** 2)

def rectangle_formula(a: float, b: float, n: int) -> float:
    """
    Составная формула прямоугольников.

    Используется формула средних прямоугольников:
        
$$I \approx h * \text{сумма } f(a + (i + 0.5) * h), i = 0, 1, \dots, n - 1$$


    Здесь:
        a, b — границы интегрирования;
        n — количество частей разбиения;
        h = (b - a) / n — шаг сетки.
    """

    h = (b - a) / n
    total = 0.0

    for i in range(n):
        x = a + (i + 0.5) * h
        total += f(x)

    return h * total

def trapezoid_formula(a: float, b: float, n: int) -> float:
    """
    Составная формула трапеций.

    Формула:
        
$$I \approx h * ((f(a) + f(b)) / 2 + \text{сумма } f(a + i*h), i = 1, 2, \dots, n - 1)$$


    Идея:
    На каждом маленьком отрезке график функции заменяется прямой линией,
    поэтому площадь считается как площадь трапеции.
    """

    h = (b - a) / n

```

```

total = (f(a) + f(b)) / 2.0

for i in range(1, n):
    x = a + i * h
    total += f(x)

return h * total

def simpson_formula(a: float, b: float, n: int) -> float:
    """
    Составная формула Симпсона.

    Формула:
        
$$I \approx h/3 * (f(x_0) + 4*f(x_1) + 2*f(x_2) + \dots + 4*f(x_{n-1}) + f(x_n))$$


    Важно:
    Для формулы Симпсона количество отрезков n обязательно должно быть
    четным.
    """

    if n % 2 != 0:
        raise ValueError("Для формулы Симпсона n должно быть четным")

    h = (b - a) / n
    total = f(a) + f(b)

    for i in range(1, n):
        x = a + i * h

        if i % 2 == 1:
            total += 4.0 * f(x)
        else:
            total += 2.0 * f(x)

    return h / 3.0 * total

@dataclass
class Method:
    """

```


Описание одного метода интегрирования.

Поля:

name — название метода;
formula — функция для вычисления интеграла;
order — порядок точности метода;
runge_sign — знак уточнения по Рунге.

По заданию:

для прямоугольников итоговое значение: I_h + погрешность;
для трапеций итоговое значение: I_h - погрешность;
для Симпсона итоговое значение: I_h - погрешность.

"""

name: str
formula: Callable[[float, float, int], float]
order: int
runge_sign: int

@dataclass

class Result:

"""

Результат работы одного метода.

Поля:

method_name — название метода;
eps — заданная точность;
n_coarse — число отрезков на грубой сетке;
n_fine — число отрезков на мелкой сетке;
h — шаг мелкой сетки;
i_2h — значение интеграла на грубой сетке;
i_h — значение интеграла на мелкой сетке;
error — оценка погрешности по правилу Рунге;
runge_integral — итоговое уточненное значение интеграла;
checks — сколько раз проверяли n.

"""

method_name: str
eps: float
n_coarse: int

```
n_fine: int
h: float
i_2h: float
i_h: float
error: float
runge_integral: float
checks: int
```

```
def parse_eps_values(args: List[str]) -> List[float]:
```

```
    """
```

```
    Получает точности из командной строки.
```

```
    Пример:
```

```
        python main.py 0.01 0.001 0.0001
```

```
    """
```

```
    if not args:
```

```
        print("Ошибка: точность нужно задать через запуск программы.")
```

```
        print()
```

```
        print("Примеры запуска:")
```

```
        print("  python main.py 0.01")
```

```
        print("  python main.py 0.01 0.001 0.0001")
```

```
        sys.exit(1)
```

```
    eps_values = []
```

```
    for value in args:
```

```
        try:
```

```
            # replace нужен, чтобы можно было ввести и 0.001, и 0,001.
```

```
            eps = float(value.replace(",", "."))
```

```
        except ValueError:
```

```
            print(f"Ошибка: значение '{value}' не является числом.")
```

```
            sys.exit(1)
```

```
    if eps <= 0:
```

```
        print(f"Ошибка: точность должна быть положительной, получено  
{value}.")
```

```
        sys.exit(1)
```

```
    eps_values.append(eps)
```

```
return eps_values
```

```
def runge_rule(method: Method, a: float, b: float, eps: float) -> Result:
```

```
    """
```

```
    Подбор n по правилу Рунге.
```

```
    Для каждого n считаются два значения:
```

```
        I_2h — значение на более грубой сетке;
```

```
        I_h  — значение на более мелкой сетке.
```

```
    Оценка погрешности:
```

```
        error = |I_h - I_2h| / (2^p - 1)
```

```
    где p — порядок точности метода:
```

```
        p = 2 для прямоугольников и трапеций;
```

```
        p = 4 для Симпсона.
```

```
    Цикл продолжается до тех пор, пока:
```

```
        error <= eps
```

```
    """
```

```
    denominator = 2 ** method.order - 1
```

```
    # Начинаем с n = 4.
```

```
    # Тогда грубая сетка имеет n/2 = 2 отрезка,
```

```
    # а для Симпсона оба значения считаются при четном количестве отрезков.
```

```
    n_fine = 4
```

```
    checks = 0
```

```
    while True:
```

```
        n_coarse = n_fine // 2
```

```
        i_2h = method.formula(a, b, n_coarse)
```

```
        i_h = method.formula(a, b, n_fine)
```

```
        error = abs(i_h - i_2h) / denominator
```

```
        runge_integral = i_h + method.runge_sign * error
```

```
        checks += 1
```

```

        if error <= eps:
            return Result(
                method_name=method.name,
                eps=eps,
                n_coarse=n_coarse,
                n_fine=n_fine,
                h=(b - a) / n_fine,
                i_2h=i_2h,
                i_h=i_h,
                error=error,
                runge_integral=runge_integral,
                checks=checks
            )

        # Берем степени двойки
        n_fine *= 2

def print_header(eps_values: List[float]) -> None:
    """
    Печатает общую информацию о задаче.
    """

    print("=" * 118)
    print("Интеграл:  $\int[0; \pi] x^2 * \exp(-x^2) dx$ ")
    print("Точности из запуска:", ", ".join(f"{eps:g}" for eps in
eps_values))
    print("=" * 118)
    print()
    print("Обозначения в таблице:")
    print("  n/2      — число отрезков на грубой сетке")
    print("  n        — число отрезков на мелкой сетке")
    print("  h        — шаг мелкой сетки")
    print("  I_2h     — значение интеграла на грубой сетке")
    print("  I_h      — значение интеграла на мелкой сетке")
    print("  Погр.    — оценка погрешности по правилу Рунге")
    print("  Итоговый I — уточненное значение интеграла")
    print()

```

```

def print_results_table(eps: float, results: List[Result]) -> None:
    """
    Печатает таблицу результатов для одной точности eps.
    """

    print("=" * 118)
    print(f"Требуемая точность eps = {eps:g}")
    print("-" * 118)

    print(
        f"{'Метод':<20}"
        f"{'n/2':>8}"
        f"{'n':>8}"
        f"{'h':>15}"
        f"{'I_2h':>18}"
        f"{'I_h':>18}"
        f"{'Погр.':>18}"
        f"{'ИТОГОВЫЙ I':>18}"
    )

    print("-" * 118)

    for result in results:
        print(
            f"{result.method_name:<20}"
            f"{result.n_coarse:>8}"
            f"{result.n_fine:>8}"
            f"{result.h:>15.10f}"
            f"{result.i_2h:>18.10f}"
            f"{result.i_h:>18.10f}"
            f"{result.error:>18.10f}"
            f"{result.runge_integral:>18.10f}"
        )

    print("-" * 118)

    best = min(results, key=lambda item: item.n_fine)
    print(
        f"Меньше всего отрезков потребовалось методу: "
        f"{best.method_name}, n = {best.n_fine}."
    )

```

```

print()

def main() -> None:
    """
    Главная функция программы.
    """

    eps_values = parse_eps_values(sys.argv[1:])

    # Границы интегрирования по варианту 18.
    a = 0.0
    b = PI

    methods = [
        Method("Прямоугольники", rectangle_formula, 2, 1),
        Method("Трапеции", trapezoid_formula, 2, -1),
        Method("Симпсон", simpson_formula, 4, -1),
    ]

    print_header(eps_values)

    for eps in eps_values:
        results = []

        for method in methods:
            result = runge_rule(method, a, b, eps)
            results.append(result)

        print_results_table(eps, results)

if __name__ == "__main__":
    main()

```